

LINUX / UNIX FUNDAMENTALS

Hands-On Lab Guide

8 Guided Labs · ~30 Minutes Each · Ubuntu Linux Edition

Companion to:

Linux / UNIX Fundamentals — 2-Day Training Deck

Platform: Ubuntu Linux (22.04 LTS or later)

Format: 1 lab per training module · self-contained · solutions included

How to Use This Guide

This guide contains one hands-on lab for each of the 8 modules in the Linux/UNIX Fundamentals training deck. Each lab is designed to run in approximately 45 minutes immediately after its matching module is taught, and stands on its own - though Labs 1–4 (Day 1) and Labs 5–8 (Day 2) build a shared set of files as the day progresses.

Environment

- A machine or VM running Ubuntu 22.04 LTS or later, with terminal access and sudo privileges.
- A non-root user account. Never run these labs as root — use sudo only where a task calls for it.
- An internet connection for the handful of apt install steps (vim, mlocate).

Conventions Used in This Guide

- Lines starting with \$ are commands you type at your normal shell prompt.
- Lines starting with sudo require administrator privileges - you'll be prompted for your password.
- Lines starting with # are comments or explanatory notes - not something you type.
- Angle brackets like <PID> mean "substitute the real value" - never type the brackets themselves.
- Each task is followed immediately by its Solution - try the task yourself first, then check your approach against it.

One-Time Setup

Run this once before Lab 1 to make sure your system is current:

```
$ sudo apt update && sudo apt upgrade -y
$ mkdir -p ~/linux-labs
```

Every lab below creates its own subfolder under ~/linux-labs, so your work stays organized and nothing from one lab overwrites another.

Lab Overview

Eight labs, one per module, each built around a realistic Northwind Retail scenario that carries over from lab to lab within a day.

Lab	Module	Focus	Time
01	Introduction to Linux	Getting Comfortable on Ubuntu	20 min
02	The Linux File System	Laying Out a Project on the File System	30 min
03	File System Security	Securing a Shared Project Directory	45 min
04	The vi Text Editor	Editing a Live Config File with vi	45 min
05	Filters & Text Processing	Triaging a Web Log with Filters	45 min
06	Regular Expressions	Hunting Failed Logins with grep	45 min
07	Processes & Task Scheduling	Taming a Runaway Process	45 min
08	Shell Scripting & Power Tools	Automating a Daily Report with sed & awk	45 min

Lab 1: Getting Comfortable on Ubuntu

Duration: ≈ 20 minutes

Objective: Log in, orient yourself on an Ubuntu system, manage your password, and use the built-in manual system to self-serve help instead of memorizing commands.

REAL-WORLD SCENARIO

You've just been issued a new Ubuntu workstation as a junior sysadmin at Northwind Retail. Before you touch any production system, your team lead wants proof that you can log in, orient yourself, and find help on your own — without asking for hand-holding on day one.

Environment Setup

\$ Open a terminal: Activities > Terminal, or press Ctrl+Alt+T on Ubuntu Desktop.
No packages required for this lab — everything used ships with Ubuntu by default.

Guided Tasks

Task 1 Confirm who and where you are.

SOLUTION

```
$ whoami
$ pwd
$ date
```

Why: whoami reports the logged-in account, pwd shows your current directory (/home/<user> right after login), and date confirms the system clock.

Task 2 Identify the system you're on.

SOLUTION

```
$ lsb_release -a
$ uname -a
$ hostname
```

Why: lsb_release -a is Ubuntu/Debian-specific and reports the release and codename; uname -a shows the running kernel - the piece every Linux distribution shares underneath its packaging.

Task 3 See the multi-user nature of Linux.

SOLUTION

```
$ who
```

```
$ w
```

Why: Even on a single-user VM, `who` and `w` show how Linux is built to track multiple concurrent logins and sessions — the same mechanism a shared server relies on.

Task 4 Change your password following secure practice.

SOLUTION

```
$ passwd
# Enter your current password, then the new one twice when prompted
```

Why: `passwd` is the standard way to rotate credentials. Coordinate with your instructor before changing this on a shared training VM - you'll need the new password to log back in.

Task 5 Self-serve documentation with `man`.

SOLUTION

```
$ man ls
/-a
# press Enter to jump to the -a flag, then q to quit
```

Why: `man` opens the full manual for any command; searching inside it with `/pattern` is faster than scrolling.

Task 6 Find a command when you only know what it should do.

SOLUTION

```
$ apropos "disk usage"
$ df -h
```

Why: `apropos` searches every `man` page description by keyword — invaluable when you know the goal but not the exact command name.

Task 7 Understand a Linux keyboard anomaly.

SOLUTION

```
$ sleep 100
# press Ctrl+C to interrupt it
$ cat
# press Ctrl+D to send end-of-input
```

Why: `Ctrl+C` sends `SIGINT` to stop a running foreground program; `Ctrl+D` signals "end of input" rather than killing anything - a classic point of confusion for newcomers.

Wrap-Up

You logged in and oriented yourself with `whoami/pwd/date`, identified the system, saw multi-user session tracking with `who/w`, secured your account with `passwd`, and practiced fishing for help with `man` and `apropos` instead of memorizing every command.

Bonus Challenge

Compare `man 1 passwd` (the command) with `man 5 passwd` (the file format of `/etc/passwd`) — see how `man` pages disambiguate a command from a file format using section numbers.

Lab 2: Laying Out a Project on the File System

Duration: ≈ 30 minutes

Objective: Navigate the file system confidently using absolute and relative paths, and create, copy, move, find, and link files like a real project setup.

REAL-WORLD SCENARIO

Northwind Retail is standing up a new internal tool called inventory-sync. You've been asked to lay out its project folder structure on the Ubuntu server and prove you can navigate, organize, and locate files without a GUI file browser.

Environment Setup

```
$ mkdir -p ~/linux-labs/lab2 && cd ~/linux-labs/lab2
```

Guided Tasks

Task 1 Build the project skeleton.

SOLUTION

```
$ mkdir -p inventory-sync/{src,logs,config,backup}
$ cd inventory-sync
$ ls -R
```

Why: Brace expansion ({src,logs,...}) creates four directories in a single mkdir call.

Task 2 Practice absolute vs. relative paths.

SOLUTION

```
$ cd src
$ cd ../config
# relative: one level up, then into config
$ cd /home/$USER
# absolute: full path from root
$ cd ~/linux-labs/lab2/inventory-sync
# ~ always means your home directory
```

Why: Relative paths depend on where you currently are; absolute paths resolve identically no matter your starting point.

Task 3 Create and populate files.

SOLUTION

```
$ cat > src/app.py << 'EOF'
print("inventory-sync starting...")
EOF
$ cat > config/settings.conf << 'EOF'
sync_interval=300
warehouse_id=NW-04
EOF
$ ls -l src config
```

Task 4 Copy, move, and rename.

SOLUTION

```
$ cp config/settings.conf backup/settings.conf.orig
$ mv config/settings.conf config/settings.conf.bak
$ ls backup config
```

Why: cp duplicates a file; mv both renames and relocates - there is no separate "rename" command in Linux.

Task 5 Clean up safely.

SOLUTION

```
$ rmdir logs
# succeeds - logs is empty
$ rmdir src
# fails: "Directory not empty"
$ rm -r src
# removes it and everything inside - use with care
```

Why: rmdir is a safety rail that refuses to delete a non-empty directory, unlike rm -r which deletes recursively without asking by default.

Task 6 Find files without knowing exactly where they are.

SOLUTION

```
$ find ~/linux-labs/lab2/inventory-sync -name "*.bak"
$ sudo apt install -y mlocate
# Ubuntu does not ship locate by default
$ sudo updatedb
$ locate app.py
```

Why: find searches live, right now, anywhere you point it; locate is instant but relies on an index (updatedb) that may be a little stale.

Task 7 Create a symbolic link.

SOLUTION

```
$ ln -s ~/linux-labs/lab2/inventory-sync/config/settings.conf.bak \
    ~/linux-labs/lab2/inventory-sync/config/latest.conf
$ ls -l config
```

Why: The l at the start of the permissions string marks latest.conf as a symbolic link pointing at the real file.

Wrap-Up

You practiced absolute vs. relative navigation, safe vs. destructive deletion (`rmdir` vs. `rm -r`), and two ways to locate files — `find` (live search) vs. `locate` (indexed search) — plus how symbolic links work.

Bonus Challenge

Run `stat settings.conf.bak` and `stat latest.conf` side by side — notice the link reports a different file type and a much smaller size than the file it points to.

Lab 3: Securing a Shared Project Directory

Duration: ≈ 45 minutes

Objective: Use groups, ownership, and permission bits to give a team controlled shared access while locking sensitive files down to a single user.

REAL-WORLD SCENARIO

Two more engineers are joining the inventory-sync project. You need to give the team shared read/write access to config/ — without letting other users on the server touch it — and lock down a secrets file that only you should be able to read.

Environment Setup (5 min)

```
$ cd ~/linux-labs/lab2/inventory-sync
$ touch config/secrets.env
```

Guided Tasks

Task 1 (4 min) — Read the current permissions.

SOLUTION

```
$ ls -l config
```

Why: Each line reads as: file type, then three rwx triads for owner, group, and others — exactly the breakdown from the training deck.

Task 2 (5 min) — Create a shared group for the team.

SOLUTION

```
$ sudo groupadd inventory-team
$ sudo usermod -aG inventory-team $USER
$ groups $USER
```

Why: usermod -aG appends the group without removing existing memberships. Log out/in (or run newgrp inventory-team) for it to apply to your current shell.

Task 3 (4 min) — Apply group ownership to the project.

SOLUTION

```
$ sudo chgrp -R inventory-team config
$ ls -l config
```

Task 4 (8 min) — Set precise permissions with chmod.

SOLUTION

```
$ chmod 770 config
# team: full access; others: none
$ chmod 660 config/settings.conf.bak
# team can read/write; not executable
$ chmod 600 config/secrets.env
# only you can read or write it
$ ls -l config
```

Why: Reading left to right, each digit is owner / group / others: 7 = rwx, 6 = rw-, 0 = ---.

Task 5 (5 min) — Cross-check numeric vs. symbolic notation.

SOLUTION

```
$ chmod u=rwx,g=rwx,o= config
# symbolic equivalent of chmod 770
$ ls -l config
```

Why: Symbolic mode (u/g/o with +/-/=) is often clearer for a single targeted change; numeric mode is faster when setting all three categories at once.

Task 6 (5 min) — Observe umask shaping new files.

SOLUTION

```
$ umask
$ touch config/newfile.txt
$ ls -l config/newfile.txt
```

Why: New files start at 666 (rw-rw-rw-) and directories at 777, minus whatever your umask subtracts — Ubuntu's default umask of 022 is why new files usually land at 644.

Wrap-Up (5 min)

You set up group-based collaboration (groupadd, usermod, chgrp) alongside individual lockdown (chmod 600), and saw how umask determines every new file's starting permissions before you ever touch chmod.

Bonus Challenge (optional, not counted in the 45 min)

Run `sudo chown nobody config/secrets.env`, then try to cat it as yourself — observe "Permission denied," and use `sudo chown $USER config/secrets.env` to take it back.

Lab 4: Editing a Live Config File with vi

Duration: ≈ 45 minutes

Objective: Edit a configuration file entirely from the keyboard using vi's modes, navigation, search, yank/put, and substitution — the way you would over a bare SSH session.

REAL-WORLD SCENARIO

The inventory-sync server has no desktop GUI and no internet access for a code-editor extension. A production config change just came in from the team lead, and the only way in is an SSH session with vi.

Environment Setup (5 min)

```
$ sudo apt install -y vim
# Ubuntu ships a minimal vim.tiny by default — install full vim for this lab
$ cd ~/linux-labs/lab2/inventory-sync/config
```

Guided Tasks

Task 1 (4 min) — Open the file and confirm your mode.

SOLUTION

```
$ vi settings.conf.bak
:set showmode
# press i, watch the mode indicator change, then Esc to return to Command mode
```

Task 2 (5 min) — Insert new configuration lines.

SOLUTION

```
G
# jump to the last line
o
# open a new line below and enter Insert mode
retry_count=3
Enter
log_level=info
Esc
```

Task 3 (4 min) — Search for a setting instead of scrolling.

SOLUTION

```
/warehouse_id
```

```
Enter
n
# repeat the search if there are multiple matches
```

Task 4 (4 min) — Yank and put a duplicate line.

SOLUTION

```
/sync_interval
Enter
yy
p
```

Why: yy yanks (copies) the current line into the unnamed register; p puts it back below the cursor.

Task 5 (5 min) — Substitute a value across the whole file.

SOLUTION

```
:%s/NW-04/NW-07/g
```

Why: % means "every line," s/old/new/ is the substitute command, and the trailing g replaces every match per line, not just the first.

Task 6 (5 min) — Undo, redo, and save.

SOLUTION

```
u
# undo the substitution
:%s/NW-04/NW-07/g
# redo by reapplying it
:wq
# save and quit
```

Task 7 (5 min) — Practice discarding an unwanted change.

SOLUTION

```
$ vi settings.conf.bak
dd
# delete a line — a "mistake"
:q!
# quit WITHOUT saving to discard it
```

Why: :q! is your safety net any time you want to walk away from changes without committing them.

Wrap-Up (5 min)

You worked through Command, Insert, and Ex (last-line) mode; practiced yank/put, search, and global substitution; and learned the critical difference between :wq (save & quit) and :q! (discard & quit).

Bonus Challenge (optional, not counted in the 45 min)

Try :set number and :set ignorecase during this session, then reopen the file and notice the settings reset — that's exactly what a personal .exrc file is for.

Lab 5: Triaging a Web Log with Filters

Duration: ≈ 45 minutes

Objective: Answer real questions about a raw access log using nothing but chained filter commands — no scripting required.

REAL-WORLD SCENARIO

Northwind's web app has been throwing errors overnight. You've been handed a raw access log and asked for quick answers before the stand-up meeting in 45 minutes — no script, just the command line.

Environment Setup (5 min)

```
$ mkdir -p ~/linux-labs/lab5 && cd ~/linux-labs/lab5
$ cat > access.log << 'EOF'
192.168.1.10 - - [14/Sep/2026:08:01:12] "GET /home HTTP/1.1" 200 512
192.168.1.15 - - [14/Sep/2026:08:02:47] "GET /login HTTP/1.1" 200 320
192.168.1.10 - - [14/Sep/2026:08:03:02] "POST /login HTTP/1.1" 401 128
192.168.1.22 - - [14/Sep/2026:08:05:10] "GET /cart HTTP/1.1" 500 256
192.168.1.15 - - [14/Sep/2026:08:06:31] "GET /checkout HTTP/1.1" 200 640
192.168.1.10 - - [14/Sep/2026:08:07:44] "POST /login HTTP/1.1" 401 128
192.168.1.30 - - [14/Sep/2026:08:09:19] "GET /home HTTP/1.1" 500 256
192.168.1.22 - - [14/Sep/2026:08:11:55] "GET /cart HTTP/1.1" 200 512
192.168.1.10 - - [14/Sep/2026:08:13:02] "GET /home HTTP/1.1" 200 512
192.168.1.15 - - [14/Sep/2026:08:14:27] "POST /checkout HTTP/1.1" 500 256
EOF
```

Guided Tasks

Task 1 (4 min) — Get a quick size check.

SOLUTION

```
$ wc -l access.log
$ wc access.log
```

Why: wc -l counts lines (requests); plain wc gives lines, words, and characters together.

Task 2 (4 min) — Preview without opening the whole file.

SOLUTION

```
$ head -n 5 access.log
$ tail -n 5 access.log
```

Task 3 (4 min) — Extract just the client IP addresses.

SOLUTION

```
$ cut -d ' ' -f1 access.log
```

Why: -d sets the delimiter (a space) and -f1 pulls out field 1 from every line.

Task 4 (5 min) — Find the busiest client.

SOLUTION

```
$ cut -d ' ' -f1 access.log | sort | uniq -c | sort -rn
```

Why: sort groups identical IPs together so uniq -c can tally them; the second sort -rn orders the tallies highest first.

Task 5 (5 min) — Isolate the HTTP status codes.

SOLUTION

```
$ cut -d '"' -f3 access.log | cut -d ' ' -f2 | sort | uniq -c
```

Why: A two-stage cut is a common pattern: split on the closing quote first, then on the space, to reach a field that isn't delimited consistently from the start of the line.

Task 6 (4 min) — Normalize text with tr.

SOLUTION

```
$ echo "get /home http/1.1" | tr 'a-z' 'A-Z'
```

Why: tr translates characters one-for-one — handy for quick case normalization before further parsing.

Task 7 (9 min) — Enrich the data by joining two files.

SOLUTION

```
$ cat > ip_owners.txt << 'EOF'
192.168.1.10 jsmith
192.168.1.15 agupta
192.168.1.22 mchen
192.168.1.30 unknown-device
EOF
$ cut -d ' ' -f1 access.log | sort -u > unique_ips.txt
$ join unique_ips.txt ip_owners.txt
```

Why: join matches lines from two files on a shared first field — both inputs must be sorted on that field first, which is why unique_ips.txt was piped through sort -u.

Wrap-Up (5 min)

You used wc/head/tail to scope the problem, cut to isolate fields, sort + uniq -c to summarize, tr to normalize text, and join to enrich data — a complete filter pipeline without writing a single script.

Bonus Challenge (optional, not counted in the 45 min)

Chain everything into one pipeline that prints only the single busiest IP and its request count.

Lab 6: Hunting Failed Logins with grep

Duration: ≈ 45 minutes

Objective: Use grep, egrep, and fgrep with regular expressions to answer a security question quickly and precisely.

REAL-WORLD SCENARIO

Security just asked: "Were there any failed login attempts last night, and from where?" You'll reuse Lab 5's access log and find the pattern with grep instead of reading every line by eye.

Environment Setup (5 min)

```
$ cd ~/linux-labs/lab5
# reuse access.log and ip_owners.txt from Lab 5
```

Guided Tasks

Task 1 (4 min) — Run a basic pattern search.

SOLUTION

```
$ grep "/login" access.log
```

Task 2 (5 min) — Anchor a pattern to the start of the line.

SOLUTION

```
$ grep "^192.168.1.10" access.log
```

Why: ^ anchors the match to the beginning of the line, so it won't accidentally match that IP appearing somewhere later in a line.

Task 3 (4 min) — Hunt for failed logins specifically.

SOLUTION

```
$ grep "401" access.log
```

Task 4 (5 min) — Combine patterns with extended regex.

SOLUTION

```
$ grep -E "401|500" access.log
# equivalently: egrep "401|500" access.log
```

Why: -E (or egrep) enables alternation with | and other extended metacharacters without backslash-escaping them.

Task 5 (6 min) — Match a range of status codes with a character class.

SOLUTION

```
$ grep -E '"[45][0-9]{2}' access.log
```

Why: [45] matches a 4 or a 5, and [0-9]{2} matches the following two digits — far more robust than listing every error code by hand.

Task 6 (5 min) — Search for a literal string, dash and all.

SOLUTION

```
$ grep -F "unknown-device" ip_owners.txt  
# equivalently: fgrep "unknown-device" ip_owners.txt
```

Why: -F (or fgrep) treats the pattern as a fixed string, not a regex — useful and faster when you want the dash to just mean a dash.

Task 7 (6 min) — Count matches instead of listing them.

SOLUTION

```
$ grep -c "401" access.log
```

Wrap-Up (5 min)

You used grep for literal and basic patterns, -E/egrep for alternation and repetition, -F/fgrep for fixed strings, and -c to count — the everyday toolkit for hunting patterns in any log file.

Bonus Challenge (optional, not counted in the 45 min)

Use `grep -v "401" access.log` to list every request that was NOT a failed login — the inverse-match flag.

Lab 7: Taming a Runaway Process

Duration: ≈ 45 minutes

Objective: Find, inspect, and safely terminate a misbehaving background process, then replace ad-hoc execution with a proper cron schedule.

REAL-WORLD SCENARIO

A background report-generation script on the Ubuntu server appears to be hung and burning CPU. You need to find it, inspect it, and kill it — then set it up to run safely and predictably on a schedule instead of being started by hand.

Environment Setup (5 min)

```
$ mkdir -p ~/linux-labs/lab7 && cd ~/linux-labs/lab7
$ cat > runaway.sh << 'EOF'
#!/bin/bash
while true; do
  echo "working..." >> /tmp/runaway.log
  sleep 2
done
EOF
$ chmod +x runaway.sh
```

Guided Tasks

Task 1 (4 min) — Start the process in the background.

SOLUTION

```
$ ./runaway.sh &
$ jobs
```

Why: & backgrounds the job immediately; jobs lists it as something like [1]+ Running.

Task 2 (5 min) — Find it among every running process.

SOLUTION

```
$ ps aux | grep runaway
$ pgrep -fl runaway.sh
```

Task 3 (4 min) — View its place in the process tree.

SOLUTION

```
$ pstree -p $USER
```

Why: Every process but init/systemd has a parent — pstree makes that lineage visible at a glance.

Task 4 (5 min) — Practice foreground/background control.

SOLUTION

```
$ fg
# brings it to the foreground
Ctrl+Z
# suspends it
$ bg
# resumes it in the background
```

Task 5 (4 min) — Watch live resource usage.

SOLUTION

```
$ top
# find bash / runaway.sh in the list and note %CPU, then press q to quit
```

Task 6 (6 min) — Terminate it cleanly and confirm.

SOLUTION

```
$ pgrep -f runaway.sh
$ kill <PID>
$ pgrep -f runaway.sh
# should return nothing
# if it won't die: kill -9 <PID>
```

Why: A plain kill sends SIGTERM, a polite request to exit; kill -9 sends SIGKILL, which the process cannot ignore.

Task 7 (7 min) — Schedule the work properly with cron.

SOLUTION

```
$ crontab -e
# add this line, then save and exit the editor:
0 7 * * * /usr/bin/date >> /home/$USER/linux-labs/lab7/cron-check.log
$ crontab -l
```

*Why: The five fields are minute, hour, day-of-month, month, and day-of-week — 0 7 * * * means "at 07:00, every day."*

Wrap-Up (5 min)

You practiced foreground/background control, found and safely killed a runaway process by PID, and replaced "start it by hand and hope" with a proper cron schedule.

Bonus Challenge (optional, not counted in the 45 min)

Use `at now + 2 minutes` to schedule a one-time `echo "hello from at"` command, then check it with `atq` before it fires.

Lab 8: Automating a Daily Report with sed & awk

Duration: ≈ 45 minutes

Objective: Redact sensitive data with sed and build a field-based summary with awk, then wrap both into a small reusable bash script.

REAL-WORLD SCENARIO

Leadership wants a one-page daily summary of the access log from Lab 5 — automatically, with internal IPs redacted before it leaves the team. You'll build a small script so nobody has to read the raw log by hand again.

Environment Setup (5 min)

```
$ cd ~/linux-labs/lab5
# reuse access.log from Lab 5
$ mkdir -p ~/linux-labs/lab8 && cd ~/linux-labs/lab8
```

Guided Tasks

Task 1 (5 min) — Redact sensitive data with sed.

SOLUTION

```
$ sed 's/192\.168\.1/REDACTED/g' ../lab5/access.log > redacted.log
$ head -3 redacted.log
```

Why: sed streams the file line by line and applies the substitution as it goes, writing the result to redacted.log without touching the original.

Task 2 (5 min) — Make an in-place edit with a safety backup.

SOLUTION

```
$ cp ../lab5/ip_owners.txt .
$ sed -i.bak 's/unknown-device/unregistered-device/' ip_owners.txt
$ diff ip_owners.txt.bak ip_owners.txt
```

Why: -i.bak edits the file in place but keeps the original as ip_owners.txt.bak — a safety net before you trust -i on its own.

Task 3 (5 min) — Build your first awk report line.

SOLUTION

```
$ awk -F'|' '{print $1, $3}' ../lab5/access.log
```

Why: The -F"" flag splits each line on double quotes; \$1 is everything before the request line, \$3 is the status/size that follows it.

Task 4 (5 min) — Count with a pattern-action pair.

SOLUTION

```
$ awk '/401|500/{count++} END{print "Errors:", count}' ../lab5/access.log
```

Why: The pattern /401|500/ selects matching lines; the action in {} runs only for those lines; END runs once after the whole file is read.

Task 5 (5 min) — Summarize requests per client IP.

SOLUTION

```
$ awk '{count[$1]++} END{for (ip in count) print ip, count[ip]}'  
../lab5/access.log
```

Why: awk's associative arrays (count[\$1]) let you tally by any field — here, by IP — without a separate sort/uniq step.

Task 6 (10 min) — Put it all together in a script.

SOLUTION

```
$ cat > daily_report.sh << 'EOF'  
#!/bin/bash  
LOG=../lab5/access.log  
echo "=== Daily Access Report ==="  
echo "Total requests: $(wc -l < "$LOG")"  
awk '/401|500/{c++} END{print "Errors:", c+0}' "$LOG"  
sed 's/192\.168\.1/REDACTED/g' "$LOG" > redacted.log  
echo "Redacted log saved to redacted.log"  
EOF  
$ chmod +x daily_report.sh  
$ ./daily_report.sh
```

Wrap-Up (5 min)

You used sed for search/replace and safe in-place edits, awk for field-based reporting and counting, and combined both into a five-line bash script — the same pattern behind real cron-scheduled reporting jobs.

Bonus Challenge (optional, not counted in the 45 min)

Add a cron entry (using what you learned in Lab 7) that runs `daily_report.sh` every morning at 6:45 AM and appends its output to a running `summary.log`.